Student Course MVC Application

Approach and Implementation Report

Technology	Spring Boot MVC, JSP, JPA/Hibernate, MySQL, Maven, JUnit, Mockito
Project URL	https://github.com/chaitanyareddy-fullstackdeveloper/Spring-Boot-MVC-Project.git
Main browser entry	http://localhost:8080/students

This report explains how the application was designed and implemented. It includes the entity relationship model, code-level operation flow, screenshots, challenges, and testing approach.

1. Architecture Overview

The application uses a layered MVC architecture. JSP pages render the browser interface, Spring MVC controllers handle request mapping and form binding, service classes enforce business rules, repositories execute persistence operations, and JPA entities map the relational model.

- **View layer:** JSP pages under `src/main/webapp/WEB-INF/jsp` use JSTL, EL, and Spring form tags.
- **Controller layer:** `StudentController` exposes create, read, and update routes.
- **Service layer:** `StudentService` and `CourseService` handle validation, duplicate email checks, and enrollment updates.
- **Repository layer:** Spring Data JPA repositories provide CRUD plus a JPQL join query.
- **Database layer:** MySQL stores students, courses, and the join table seeded by `data.sql`.

2. Entity Relationship Design

A student can enroll in many courses, and each course can contain many students. A normalized join table named `student_courses` stores the association using two foreign keys.

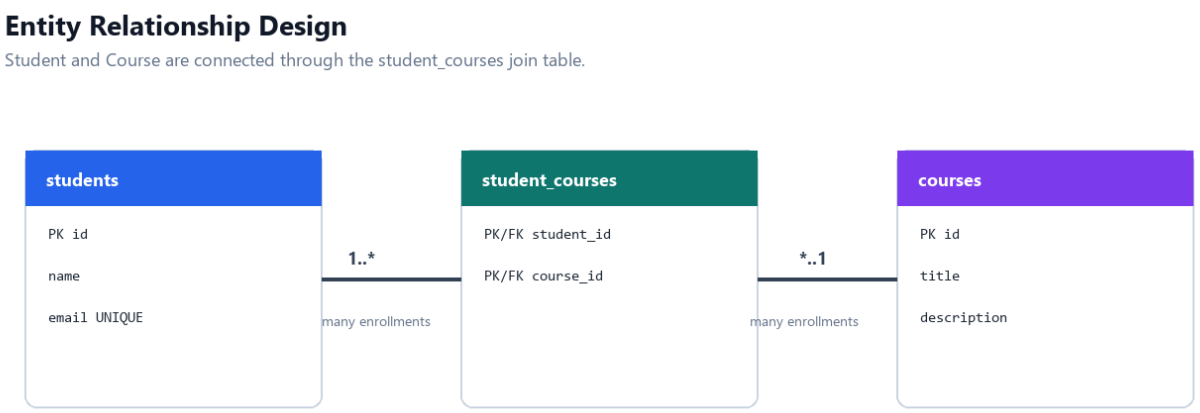

Figure 1: Many-to-many database design.

Entity: Many-to-Many Mapping

```java
@Entity
@Table(name = "students",
       uniqueConstraints = @UniqueConstraint(name = "uk_students_email", columnNames = "email"))
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 150)
    private String email;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "student_courses",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id"))
    private Set<Course> courses = new HashSet<>();
}
```

Figure 2: Entity mapping code excerpt.

3. Read Operation

The list page is served by `GET /students`. The controller loads students with their enrolled courses and also sends rows from the custom JPQL inner join query so the page can demonstrate the join output.

Repository: JPQL Join Query

```java
public interface StudentRepository extends JpaRepository<Student, Long> {
    boolean existsByEmail(String email);
    boolean existsByEmailAndIdNot(String email, Long id);

    @EntityGraph(attributePaths = "courses")
    @Query("SELECT DISTINCT s FROM Student s ORDER BY s.name")
    List<Student> findAllWithCourses();

    @Query("SELECT s.name AS studentName, c.title AS courseTitle " +
           "FROM Student s JOIN s.courses c ORDER BY s.name, c.title")
    List<StudentCourseView> findStudentCoursePairs();
}
```

Figure 3: Repository read methods and JPQL inner join.

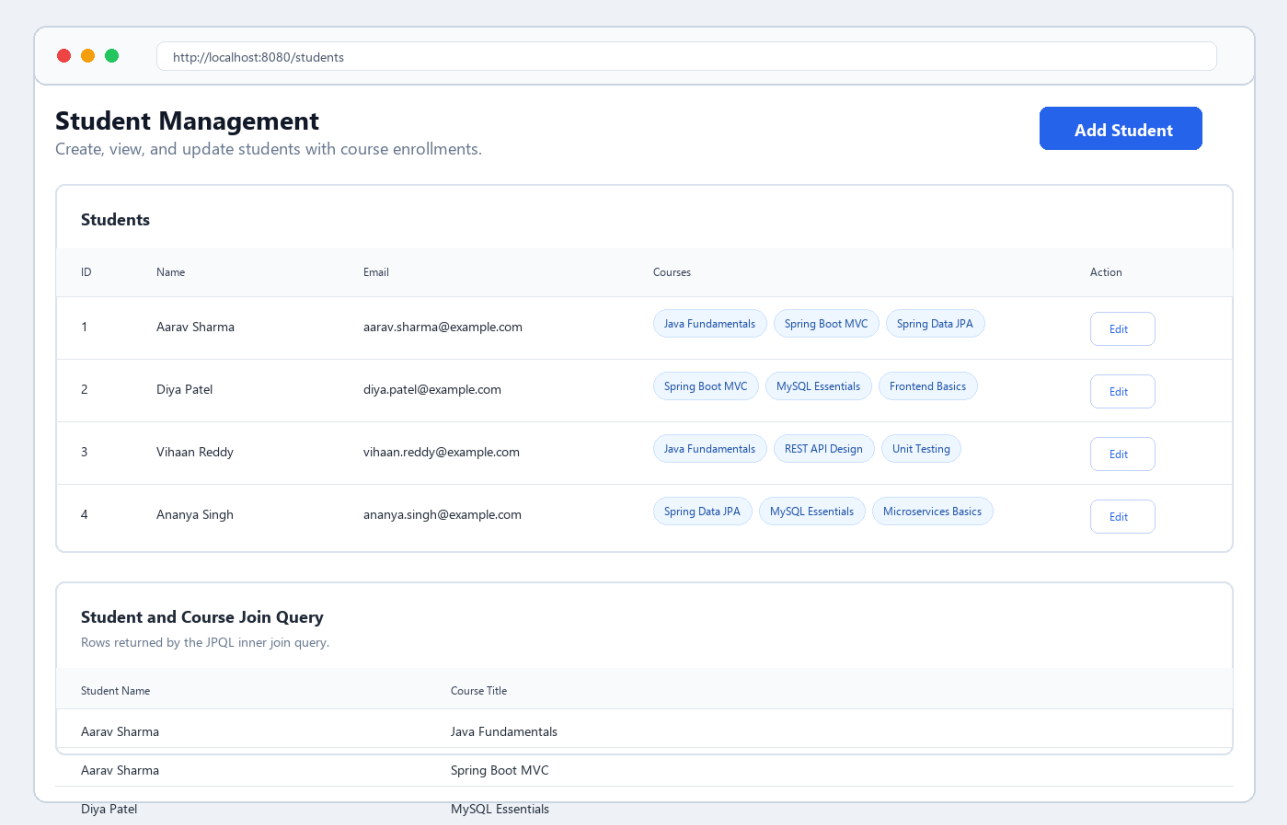

Figure 4: Student list and join-query result UI. Screenshot-style rendering based on the JSP page design and seed data.

4. Create Operation

The create flow starts at `GET /students/new`. The JSP binds to `StudentForm`, validates name and email, accepts selected course IDs, and submits to `POST /students`. The service normalizes the email, checks uniqueness, loads selected courses, and saves the student.

Service: Create and Update Business Rules

```
@Transactional
public Student createStudent(StudentForm form) {
    String normalizedEmail = normalizeEmail(form.getEmail());
    if (studentRepository.existsByEmail(normalizedEmail)) {
        throw new DuplicateEmailException(normalizedEmail);
    }
    Student student = new Student();
    applyForm(student, form);
    return studentRepository.save(student);
}

@Transactional
public Student updateStudent(StudentForm form) {
    Student student = studentRepository.findByIdWithCourses(form.getId())
        .orElseThrow(() -> new ResourceNotFoundException("Student not found with id: " + form.getId()));
    if (studentRepository.existsByEmailAndIdNot(normalizeEmail(form.getEmail()), form.getId())) {
        throw new DuplicateEmailException(normalizeEmail(form.getEmail()));
    }
    applyForm(student, form);
    return studentRepository.save(student);
}
```

Figure 5: Create and update service logic.

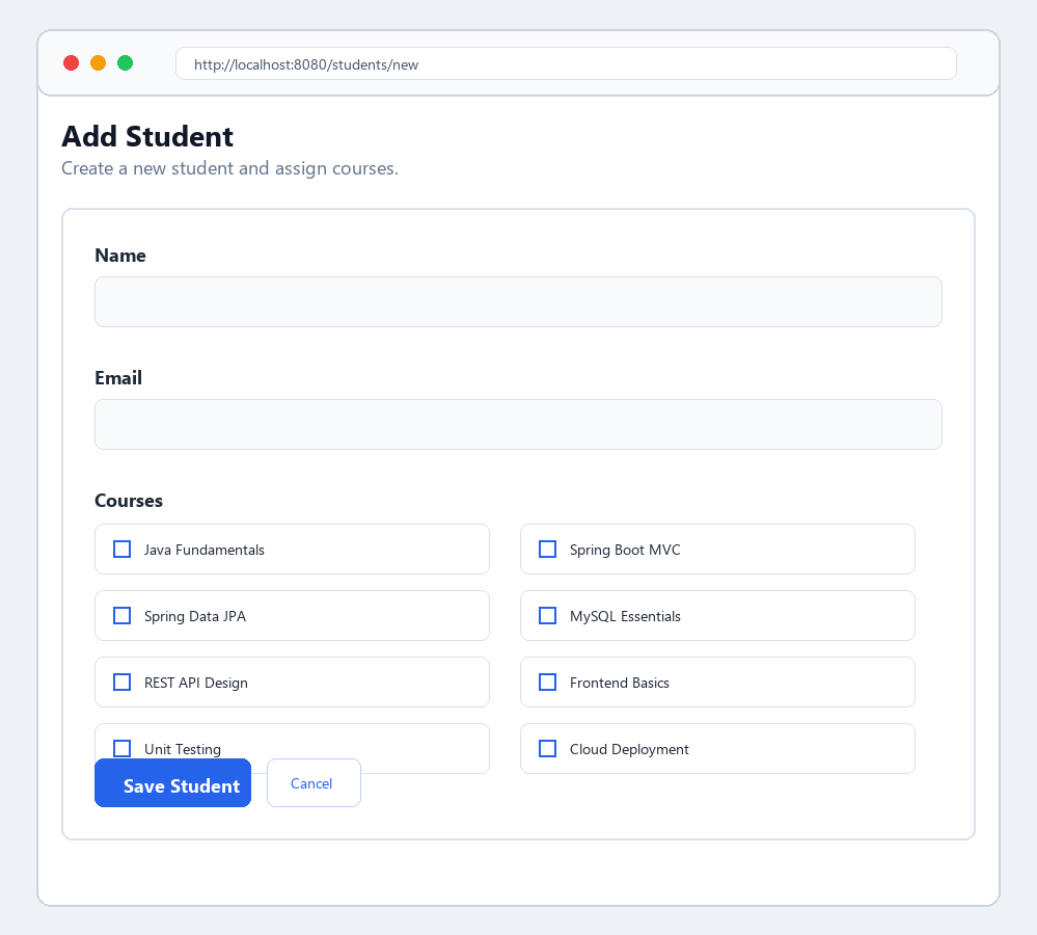

Figure 6: Add student form. Screenshot-style rendering based on the JSP page design.

5. Update Operation

The update flow starts from the Edit button on the list page. `GET /students/edit/{id}` loads the existing student with courses and converts it into `StudentForm`. `POST /students/update` validates input, prevents duplicate email conflicts with other students, replaces the selected course set, and saves the entity.

Controller: MVC Form Flow

```
@GetMapping
public String listStudents(Model model) {
    model.addAttribute("students", studentService.getAllStudents());
    model.addAttribute("studentCoursePairs", studentService.getStudentCoursePairs());
    return "students";
}

@PostMapping
public String saveStudent(@Valid @ModelAttribute("studentForm") StudentForm studentForm,
                          BindingResult bindingResult,
                          Model model,
                          RedirectAttributes redirectAttributes) {
    if (bindingResult.hasErrors()) {
        model.addAttribute("courses", courseService.getAllCourses());
        return "add-student";
    }
    studentService.createStudent(studentForm);
    redirectAttributes.addFlashAttribute("successMessage", "Student created successfully.");
    return "redirect:/students";
}
```

Figure 7: Controller methods for list and create flow; update uses the same validation pattern.

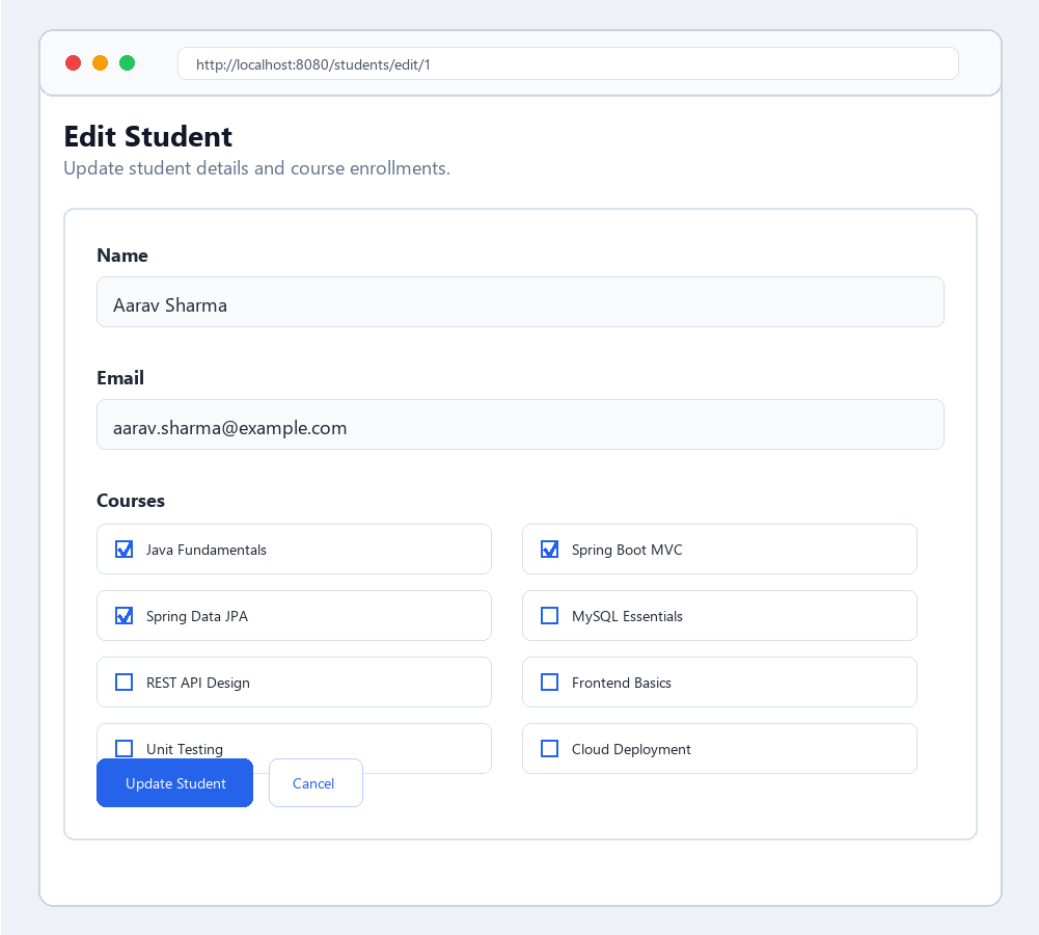

Figure 8: Edit student form with pre-selected courses. Screenshot-style rendering based on the JSP page design.

6. Exception Handling and Validation

Validation is split between Bean Validation annotations and service-level business checks. Form errors return the same JSP with messages beside the fields. Duplicate emails are handled by a custom exception and database constraint, while missing students or courses are reported through `ResourceNotFoundException` and `@ControllerAdvice`.

Case	How it is handled
Blank or invalid fields	Bean Validation annotations and Spring form error rendering
Duplicate email	Service check plus unique database constraint; message returned to the form
Missing student/course	ResourceNotFoundException handled globally and redirected safely
Database integrity issue	DataIntegrityViolationException handled by ControllerAdvice

7. Testing Approach

The repository test verifies the JPQL join and eager loading behavior with an H2 test database. Service tests use Mockito to verify duplicate-email checks, course selection, update behavior, and missing-resource handling.

Testing: Repository Join Verification

```java
@DataJpaTest
@ActiveProfiles("test")
class StudentRepositoryTest {
    @Test
    void findStudentCoursePairsReturnsJoinedStudentAndCourseTitles() {
        Course java = courseRepository.save(new Course("Java Fundamentals", "Learn Java basics."));
        Student student = new Student("Ananya Singh", "ananya@example.com");
        student.getCourses().add(java);
        studentRepository.saveAndFlush(student);

        List<StudentCourseView> rows = studentRepository.findStudentCoursePairs();

        assertThat(rows).extracting(row -> row.getStudentName() + " - " + row.getCourseTitle())
            .contains("Ananya Singh - Java Fundamentals");
    }
}
```

Figure 9: Repository test excerpt.

Verification result: `mvn test` completed successfully with 8 tests passed. `mvn package` also completed successfully.

8. Challenges Faced and Solutions

- **Many-to-many form binding:** JSP checkboxes submit course IDs, not Course objects. I solved this with a dedicated `StudentForm` DTO and service conversion through `CourseService.getCoursesByIds()`.
- **Lazy loading in JSP:** Rendering course chips after the transaction can cause lazy-loading errors. I used `@EntityGraph(attributePaths = "courses")` on read queries.
- **Duplicate email safety:** I combined a service-level uniqueness check with a database unique constraint for defense in depth.
- **Spring Boot 3 JSP/JSTL compatibility:** JSP pages use Jakarta JSTL tag URIs and the Jasper dependency required by embedded Tomcat.
- **Local database availability:** MySQL is required for runtime, but no MySQL service was available on this machine during documentation. Tests were verified through H2, and the PDF UI figures were rendered from the JSP design and seed data rather than from a live MySQL session.

9. Run Instructions

Install or start MySQL, then configure credentials through environment variables if your database password differs from the default.

Step	Command or URL
Set credentials	DB_USERNAME=root, DB_PASSWORD=your_mysql_password
Run app	mvn spring-boot:run
Open browser	http://localhost:8080/students
Run tests	mvn test

GitHub URL: https://github.com/chaitanyareddy-fullstackdeveloper/Spring-Boot-MVC-Project.git